

目 录

一、AI 编程的结构性困境	3
缺陷一：上下文不足	3
缺陷二：流程缺失	4
为什么模型越强，问题越严重？	4
二、Manon 的解决方案	4
设计哲学	4
系统架构	4
三、知识图谱 — 核心基础	5
索引内容	5
混合检索机制	6
语言支持	6
四、四大查询工具	6
4.1 manon_search — 语义代码搜索	6
4.2 manon_graph — 调用图遍历	7
4.3 manon_deep_query — 多轮深度查询	7
4.4 manon_impact — 提交影响分析	8
四工具综合效能	9
五、五大开发技能	9
5.1 /idea — 需求精炼	10
5.2 开发阶段 — Claude + 图谱	10

5.3 /exp — 体验验证

10

5.4 /tc — 测试覆盖

11

5.5 /dao — 代码简化

11

技能自增强循环

12

六、代码健康评估

12

七、自动化 Hooks

13

八、安全与隐私

13

九、实测效能总览

14

代码理解效率

14

开发效率（自我验证）

14

十、兼容性

14

支持的 IDE

14

系统要求

14

十一、价格方案

15

你的投资回报

15

快速开始

15

Manon — AI 架构师，为你的代码而生

一、AI 编程的结构性困境

AI 编程正在改变软件开发，但当前的 AI 编程工具存在两个根本性缺陷，使得 AI 无法真正可靠地完成复杂开发任务。

缺陷一：上下文不足

AI 模型在编写代码时，无法看到完整的调用图、依赖链和模块边界。它只能基于当前打开的文件和有限的上下文进行推理，导致：

- **幻觉** — 猜测函数之间的调用关系，猜错就引入 bug
- **遗漏** — 修改一个函数却不知道有 20 个调用方会受影响
- **重复** — 不知道项目中已有类似功能，重新实现一遍

这个问题在三个层面都会造成失败：

层面	失败模式	具体表现
架构层	看不到全局做结构决策	不必要的抽象层、过度模块化、过早泛化、配置/事件系统过度设计
模块层	不理解依赖关系就划分边界	单模块功能膨胀、职责不清、跨模块重复、过度耦合
代码层	不知道谁调用谁就修改代码	死代码残留、引入循环依赖、函数拆分过细、内聚性低

缺陷二：流程缺失

AI 模型倾向于跳过需求分析、测试和验证，直接编写代码。没有结构化的开发流程，导致：

- **范围蠕变** — 需求不明确就开始写代码，越写越偏
- **未测试的代码** — 写完功能不补测试，上线后出 bug
- **静默回归** — 修改代码后没有验证，破坏了已有功能

为什么模型越强，问题越严重？

- **AI 无法"感知"架构意图** — 它局部优化，产出技术上正确但违反系统设计的代码
- **AI 生成速度远超验证速度** — 没有图谱验证，错误模式以机器速度扩散
- **每次 AI 会话都是从零开始** — 没有对过去决策的记忆，同样的结构错误反复出现

强模型 + 差上下文 + 无流程 = 自信的垃圾，速度更快。

二、Manon 的解决方案

Manon 从根源解决这两个问题：通过**知识图谱**补全上下文，通过**开发技能规范**流程。

设计哲学

Manon 的核心设计原则是 "**让 AI 基于代码事实做决策，而非基于想象**"。每一个功能都围绕这个原则构建：

1. **代码留在本地** — 只上传 AST 解析数据，源代码永远不离开开发者机器
2. **增量同步** — 文件哈希检测变更，只传差异，不重复上传
3. **混合检索** — 图遍历（结构性）+ 向量搜索（语义性），精准获取相关代码
4. **技能只做 AI 做不好的事** — 如果纯对话能完成，就不需要技能

系统架构

本地 (manon_mcp)

组件	职责
IDE 集成	通过 MCP 协议与 Claude Code / Cursor 等连接
tools/	MCP 工具处理器
_client	HTTP 客户端，连接云端 SaaS API
_sync	文件扫描 + 批量上传
_hooks	git pre-push + 编辑器 hooks
core/ast	tree-sitter AST 解析
codeindex/	各语言专用解析器

云端 (saas)

组件	职责
FastAPI 应用	REST API 服务
路由层	查询 / 索引 / 仓库管理
CodeGraph	NetworkX 有向图存储
VectorIndex	numpy 余弦相似度检索
pipeline	AST 数据 -> 图谱构建
impact	提交影响分析引擎
llm.py	深度查询编排
embedding	向量生成服务

数据流向：本地扫描文件 -> 解析 AST -> 上传变更 -> 云端构建图谱 + 向量 -> 查询结果返回本地

三、知识图谱 — 核心基础

知识图谱是 Manon 的核心基础。它索引代码库中的每一个函数、类、方法、调用关系、导入链和模块边界，构建一个完整的代码结构图。

索引内容

Manon 通过 tree-sitter 进行 AST 级别的代码解析，提取以下结构信息：

提取项	说明
实体 (Entities)	函数、类、方法、模块、变量
关系 (Relations)	调用 (calls)、导入 (imports)、继承 (inherits)、处理 (handles)
代码块 (Chunks)	每个实体的代码片段，用于向量检索
HTTP 路由	Flask/FastAPI、Express、NestJS 的路由注册（HANDLES 边类型）

混合检索机制

Manon 的检索不是简单的关键字搜索，而是**向量搜索 + 图遍历**的混合模式：

- 向量搜索**：将查询转为向量嵌入，与代码块向量计算余弦相似度，找到语义最相关的实体
- 图扩展**：沿图谱边展开，自动关联调用方、被调用方、同模块实体
- 结果融合**：返回实体 + 关系 + 代码片段的结构化结果

这使得 Manon 能够回答"描述性"的问题（不知道关键字，用自然语言描述意图），而传统的 Grep 只能搜索已知关键字。

语言支持

深度解析（符号 + 调用 + 导入 + 继承 + 路由提取）：

Python, TypeScript, JavaScript, Java, PHP

通用解析（符号 + 导入，基于 tree-sitter 自动下载）：

Go, Rust, C, C++, C#, Ruby, Swift, Kotlin, Scala, Lua, R, Elixir, Dart, Haskell, OCaml, Bash, Zig

四、四大查询工具

Manon 提供四个查询工具，覆盖从代码搜索到风险评估的完整场景。

4.1 manon_search — 语义代码搜索

解决的问题：不知道关键字时，如何找到相关代码？

工作原理：自然语言 → 向量嵌入 → 检索最相似实体 → 沿图谱边扩展关联代码

典型场景：

- "处理用户认证的函数在哪里？"
- "数据库连接池是怎么管理的？"
- "WebSocket 消息路由逻辑"

对比原生工具的优势：

维度	manon_search	Grep
查询方式	自然语言描述意图	必须知道精确关键字
返回内容	实体 + 关系 + 代码片段	匹配的文本行
跨模块关联	自动关联调用方/被调用方	每个文件独立，需手动跳转
工具调用次数	1 次	平均 7.2 次
质量评分	4.2/5	2.6/5

4.2 manon_graph — 调用图遍历

解决的问题：修改一个函数前，如何知道谁在调用它？改了之后会影响什么？

工作原理：给定符号名 → 在图谱中定位 → 按方向遍历调用边 → 返回多层调用链

参数：

- `symbol`：函数/类名
- `direction`：`callers`（谁调用我） / `callees`（我调用谁） / `both`（双向）
- `depth`：遍历深度（1-3 层）

关键能力：Grep 无法区分"调用方"和"被调用方"——搜索一个函数名会返回所有出现位置，无法判断方向。manon_graph 提供**有方向的**调用关系。

实测对比：

维度	manon_graph	原生工具
工具调用次数	1 次	平均 9.8 次
质量评分	4.6/5	2.6/5
核心优势	有方向的调用链、多层深度	无法区分调用方向

4.3 manon_deep_query — 多轮深度查询

解决的问题：跨模块的架构问题，需要多次查询才能拼出完整图景。

工作原理：

- 1. 用户提出架构级问题
- 2. 服务端 LLM 分析已有上下文，识别信息缺口
- 3. 自动生成补充查询，在图谱中搜索
- 4. 重复直到所有子问题都被覆盖

典型场景：

- "项目的认证流程是怎样的？从前端到数据库经过哪些模块？"
- "这个项目的多通道集成是如何设计的？各通道之间有什么依赖？"

关键价值：一次 MCP 调用，服务端自动完成所有迭代。不需要人工一轮轮提问。

实测对比：

维度	manon_deep_query	原生工具
工具调用次数	1 次	平均 17.6 次
Token 消耗	~8K	~73K
质量评分	4.6/5	2.6/5
平均迭代轮数	2.25 轮（自动完成）	需人工多轮查询

4.4 manon_impact — 提交影响分析

解决的问题：提交代码后，如何快速评估这个提交的风险？

工作原理：解析 git diff → 提取变更符号 → 在图谱中追溯 2 跳调用方 → 计算风险分（0-100）

返回内容：

- 变更的符号列表（函数、类）
- 直接调用方
- 间接调用方（2 跳）
- 受影响的模块
- 受影响的测试
- 传播链可视化
- 量化风险评分

应用场景：

- 每次 commit 后自动运行（通过 hook）
- CI/CD 流水线中作为质量门禁
- 代码审查的辅助工具

实测对比：

维度	manon_impact	原生工具
工具调用次数	1 次	平均 20 次
Token 消耗	~3.5K	~232K（节省 98.5%）
耗时	~5 秒	~110 秒

四工具综合效能

指标	Manon	原生工具	提升
平均工具调用	1 次	13.7 次	减少 91%
总 Token 消耗	~19.5K	~350K	节省 94%
平均质量评分	4.3/5	3.2/5	+34%

五、五大开发技能

Manon 的技能系统覆盖完整的开发生命周期。每个技能都以知识图谱为支撑，确保决策基于代码事实。

技能只在 AI 纯对话无法完成时才存在——它们要么需要外部工具集成（图谱 API、覆盖率数据、Playwright），要么需要确定性工作流，要么需要结构化输出。

顺序	阶段	技能	输入	输出
1	需求精炼	/idea	模糊需求	开发文档
2	编码开发	Claude + 图谱	开发文档	功能代码
3	体验验证	/exp	功能代码	Bug 报告
4	测试补充	/tc	验证通过的代码	测试用例
5	代码治理	/dao	完整代码	简化后的代码

全部阶段基于知识图谱，形成闭环。

5.1 /idea — 需求精炼

做什么：将模糊需求变成可执行的开发文档。

为什么需要技能：普通对话只能基于通用知识提问。/idea 基于图谱事实提问——"模块 X 的扇入值为 12，新功能应该放在这里还是新建模块？"

完整流程：

阶段	动作	基于图谱的优势
CONTEXT	查询图谱 + GitHub，收集相关模块和依赖	15 个相关模块 + 3 个图谱实体 + 健康评分，一次脚本调用完成
EXPLORE	基于代码事实提出苏格拉底式问题（3-7 轮）	问题基于扇入值、模块边界等具体数据，不是通用问题
PROPOSE	生成 2-3 个技术方案，每个附带影响分析	每个方案的影响范围由图谱计算，不是猜测
DOCUMENT	输出完整开发文档，通过 5 维自动审查	文档与实际代码结构一致

5.2 开发阶段 — Claude + 图谱

做什么：在 IDE 中正常写代码，Manon 通过 hooks 自动介入。

自动化机制：

- 搜索代码前（PreToolUse hook）：在使用 Grep/Glob 前，提醒先查询图谱，避免低效的关键字搜索
- 提交代码后（PostToolUse hook）：每次 `git commit` 后自动运行 `manon_impact`，即时评估风险
- 推送代码后（git pre-push hook）：自动增量更新图谱 + 输出健康评分变化

5.3 /exp — 体验验证

做什么：AI 像真实用户一样操作你的产品，发现真实 bug。

为什么需要技能：AI 凭想象验证功能会遗漏真实问题。/exp 使用 Playwright（Web）或 Bash（CLI/Service）实际操作产品。

支持的产品类型：

类型	工具	适用场景
web	Playwright MCP	前端页面 — 点击、输入、导航
cli	Bash	命令行工具 — 执行命令、检查输出
service	curl + 日志	后端 API — 发请求、验证响应
hybrid	Playwright + Bash	前后端联动 — 全链路验证

验证循环：最多 3 轮 —— 操作产品 → 发现 bug → 修复 → 重测

实测效果（在 /idea 技能发布前）：

轮次	结果	发现的 Bug
第 1 轮	4/5 通过	API 响应解析错误；编码崩溃
第 2 轮	0/1 通过	符号过滤器过严；中文查询崩溃
第 3 轮	1/1 通过	—

3 轮验证，发现并修复 4 个真实 bug — 这些 bug 如果没有 /exp，就会直接发布给用户。

5.4 /tc — 测试覆盖

做什么：按结构重要性而非随机顺序补充测试。

为什么需要技能：普通 AI 补测试是随机的。/tc 查询图谱中每个实体的扇入值、复杂度和中心度，优先测试结构上最重要的代码。

完整流程：

步骤	动作
1. 覆盖率扫描	读取 coverage 数据，识别未覆盖代码
2. 图谱优先级	按扇入值 x 复杂度 x 中心度排序
3. 编写测试	对高优先级实体生成测试
4. 运行验证	执行测试，确认通过
5. 提交	自动 commit

5.5 /dao — 代码简化

做什么：扫描代码库，自动识别和修复技术债。

为什么需要技能：代码简化需要图谱验证。例如删除死代码前，必须确认在图谱中确实零调用方，而不是因为工具遗漏。

完整流程：

步骤	动作
1. 健康扫描	运行 8 维代码健康评估
2. 三层分类	将问题分为 Architecture / Module / Code 三层
3. A/M 面板	架构和模块级问题展示给开发者选择处理
4. C 级自动修复	代码级问题（死代码、过度间接等）自动修复
5. 图谱验证	所有修复在提交前经图谱验证

实测效果（在 Manon 自身代码库上）：

指标	修复前	修复后
健康评分	88/100	97/100
死代码实体	47 个	29 个 (-38%)
测试覆盖率	32%	61% (+29pp)

技能自增强循环

五个技能形成一个闭环：/idea 定义需求 → 基于图谱写代码 → /exp 端到端验证 → /tc 补充测试 → /dao 清理复杂度 → 发现的问题反馈到下一个循环。

Manon v1.0 到 v1.2.4 的全部开发都通过这个循环完成。

六、代码健康评估

Manon 从 8 个维度评估代码库的结构健康，每个维度 10 分，加权汇总为 100 分总分。

缩写	维度	衡量什么	权重
MC	模块耦合度	跨模块依赖占总依赖的比例	15%
CD	循环依赖	模块间循环依赖数量	10%
FI	扇入集中度	被过多模块调用的实体比例	15%
DC	死代码	零调用方的实体比例	10%
FS	函数复杂度	超大函数的比例	10%
TD	技术债务	TODO/FIXME 注释密度	15%
MF	模块碎片化	过小模块 + 过深路径比例	15%
RE	间接层密度	桶式重导出的比例	10%

这 8 个维度都是**图谱原生**的 —— 直接从知识图谱的结构数据中计算，不需要额外的静态分析工具。

七、自动化 Hooks

Manon 安装后自动配置三种 hook，无需手动管理：

Hook	触发时机	做什么	价值
git pre-push	<code>git push</code> 后	增量更新图谱 + 输出健康评分变化	图谱始终保持最新
PreToolUse	Grep/Glob/Agent 前	提醒先查询图谱	减少低效的关键字搜索
PostToolUse	<code>git commit</code> 后	自动运行 <code>manon_impact</code>	每次提交即时评估风险

这意味着开发者不需要刻意使用 Manon — 它自动融入现有 workflow。

八、安全与隐私

Manon 的安全设计原则是**最小化数据传输**：

安全特性	说明
源代码不上传	只上传 AST 解析数据（函数名、类名、参数签名、调用关系），源代码永远留在本地
增量同步	基于文件 SHA-256 哈希检测变更，只传差异数据
租户隔离	每个用户的图谱数据完全隔离，互不可见
API Key 认证	每次安装生成独立的 API Key
支持私有部署	Enterprise 客户可选择自托管方案，数据完全不出内网

九、实测效能总览

代码理解效率

在 2,100 文件的大型项目上实测：

维度	Manon	原生工具	差距
完成项目分析的耗时	~30 分钟	~8-12 小时	快 16-24 倍
分析准确率	95%+	60-70%	+30%
工具调用次数	1 次/任务	13.7 次/任务	减少 91%
Token 消耗	~19.5K	~350K	节省 94%
质量评分	4.3/5	3.2/5	+34%

开发效率（自我验证）

Manon 使用自己的技能开发自己，以下是真实产出：

指标	改进前	改进后	变化
代码健康分	88/100	97/100	+9
死代码实体	47	29	-38%
测试覆盖率	32%	61%	+29pp
知识图谱关系数	600	1053	+74%

十、兼容性

支持的 IDE

Claude Code, Cursor, Windsurf, Zed, Continue, CodeBuddy

系统要求

- Python 3.10+（Windows 下安装器自动通过 winget 安装）
- 网络连接
- macOS / Linux / Windows

十一、价格方案

简单透明，按席位订阅。一次安装 = 一个 API Key = 一个席位。

	Free	Pro	Enterprise
价格	免费	¥399/月/席位	¥999/月/席位
有效期	注册后 30 天	订阅制	订阅制
仓库数	1 个	5 个	无限
全部功能	全部开放	全部开放	全部开放
支持	社区	优先	专属 + 私有部署

你的投资回报

一个月薪 3 万的开发者，Manon 帮他节省约 30% 的代码理解和质量维护时间
= 每月节省约 9,000 元。

Pro 订阅 ¥399/月，不到节省成本的 5%。

30 天免费试用，全功能无限制。

快速开始

只需三步：

第一步：安装（一行命令）

BASH

```
git clone https://github.com/brandonzyy/manon.git  
cd manon && bash install.sh
```

安装器自动完成所有配置：检测编辑器、安装依赖、注册账号、配置 MCP 连接、安装 hooks 和全部技能。安装完成后重启编辑器。

第二步：激活（一个命令）

在编辑器中输入 `/manon`。Manon 自动分析目录结构、扫描源码、构建知识图谱、输出健康报告。通常 1-3 分钟完成。

第三步：无需操作

激活后不需要刻意做任何事。Manon 通过 hooks 自动融入你的工作流：

- 你搜索代码时 — Claude 自动调用图谱查询，获取更精准的上下文
- 你提交代码时 — 自动运行影响分析，评估风险
- 你推送代码时 — 自动增量更新图谱，保持同步

需要结构化工作流时，输入 `/idea`、`/exp`、`/tc`、`/dao` 即可，按提示操作。

[GitHub](#) · [问题反馈](#) · [讨论社区](#)